

GATE 2026

DBMS

CS/IT

Complete Handwritten Notes



Lahori Venkatesh
Full-Stack Developer
MNIT Jaipur



Database:- The collection of data, usually referred to as the database, contains information relevant to an enterprise.

Database Management System:- The database management system (DBMS) is a collection of interrelated data and a set of programs to access those data.

→ info retrieval,
insertion, deletion,
updatation.

Goal of DBMS

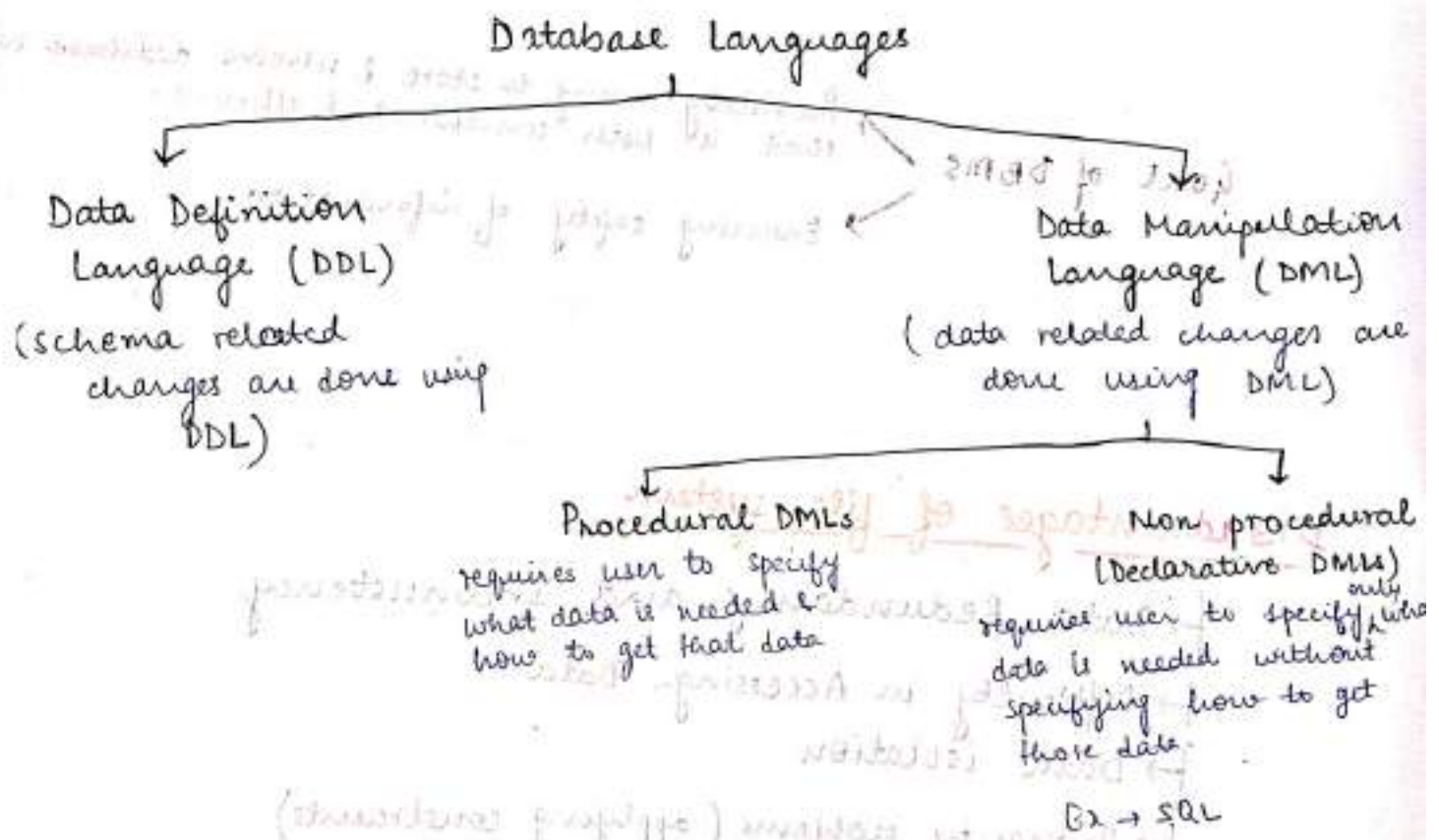
- Providing a way to store & retrieve database info that is both convenient & efficient.
- Ensuring safety of information.

Disadvantages of file system

- Data Redundancy and Inconsistency
- Difficulty in Accessing Data
- Data isolation
- Integrity problems (applying constraints)
- Atomicity problem
- Concurrent - Access Anomalies
- Security problems

Instance - The collection of information stored in the database at a particular moment is called an instance of database.

Schema - The overall design of the database is called database schema.



View of data -

1. Physical level -

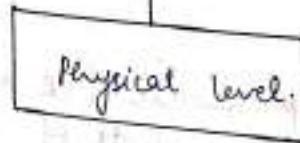
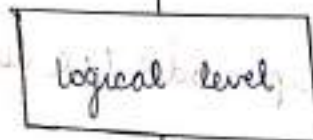
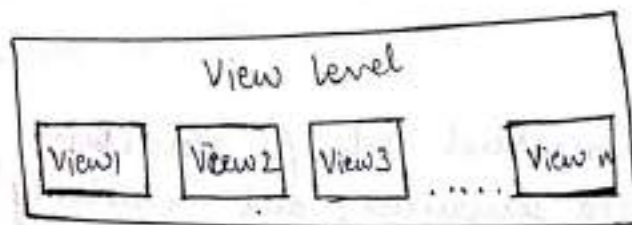
how data is actually stored in physical memory.

2. Logical level -

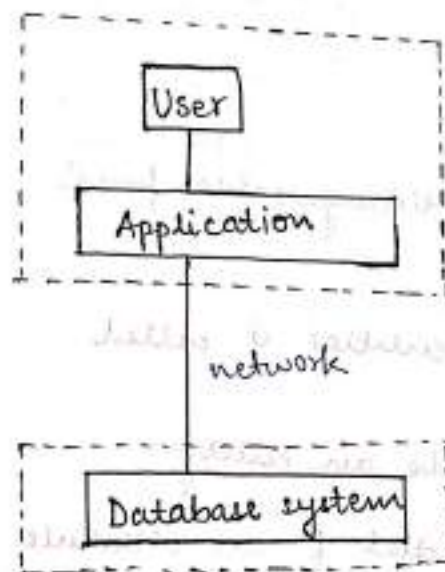
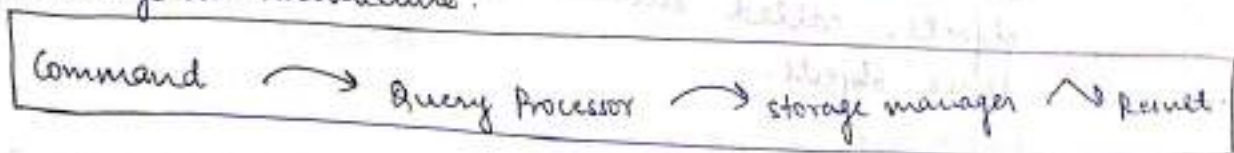
tables, at this level, we see data distributed in the form of tables in database.

3. View level - defines which part of database can be viewed by a person.

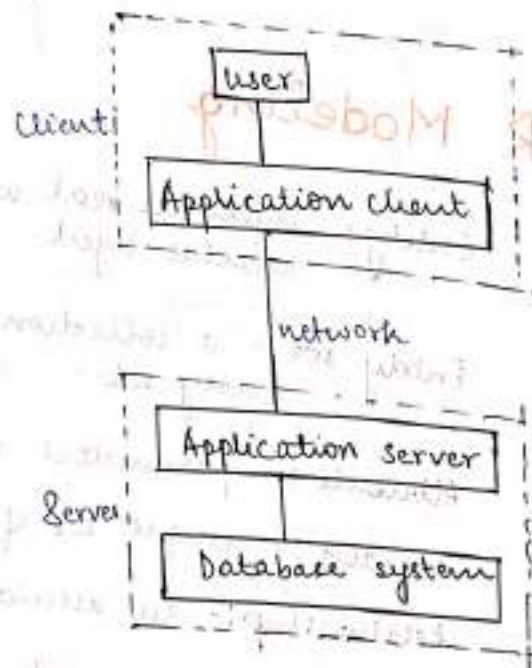
manager can view all the tables
employee can view only his record.



Database System Architecture.



2 tier architecture



3 tier architecture

Data Models

A collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints.

2 models

- Entity-Relationship model (ER model)
- Relational model

1. Entity Relationship model (ER model)

ER data model consists of a collection of basic objects, called entities, and of relationships among these objects.

2. Relational model

uses a collection of tables to represent both data & relationships.

E-R Modeling

Entity:- object in real world that is distinguishable from another object.

Entity set:- a collection of similar entities is called entity set.

Attribute:- parameters which describe an entity

Domain:- unique set of values permitted for an attribute

Relationship:- an association among two or more entities

Relationship-set - a set of similar relationships

Key - an attribute or a set of attributes whose values uniquely identify an entity in a set.

Discriptive attribute - attribute of a relationship.

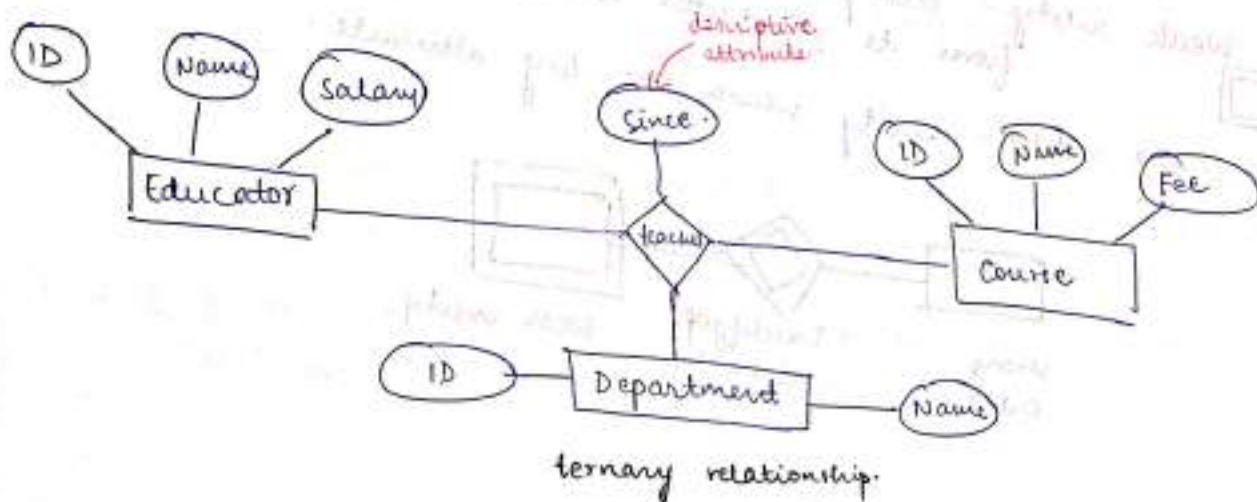
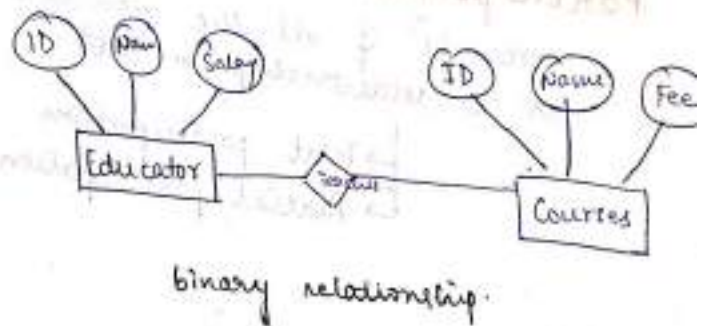
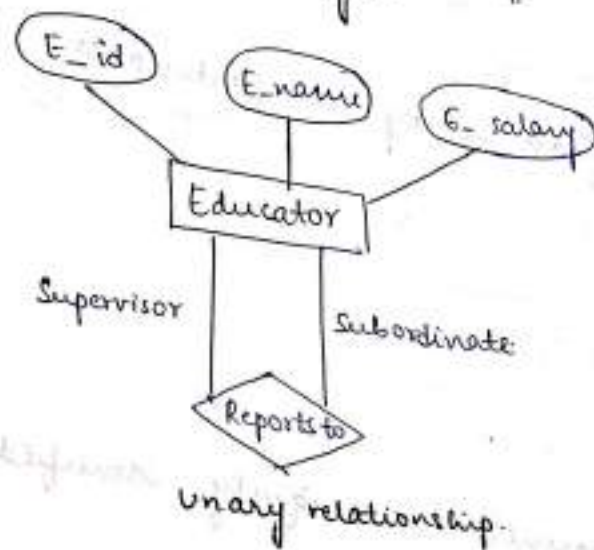
Entity set $\rightarrow$ 

Relationship $\rightarrow$ 

attribute $\rightarrow$ 

Types of Relationships -

1. Unary - Relationship b/w entities of same entity set.
2. Binary - " " " " 2 entity sets.
3. Ternary - " " " " 3 entity sets.



Mapping Cardinality

The maximum number of ~~or~~ relationships in which an entity can participate is ~~also~~ called mapping cardinality.

- One to one relationship
- One to many relationship
- Many to one relationship
- Many to many relationship

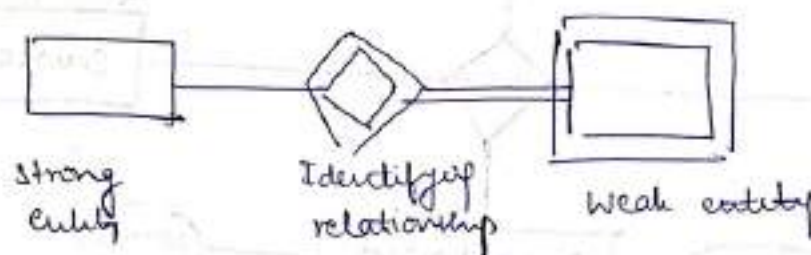
Participation constraints

represents if all the members of an entity-set participate in a relationship or not.

- Total participation
- Partial participation

Weak entity - entity which cannot be uniquely identified from its attributes alone.

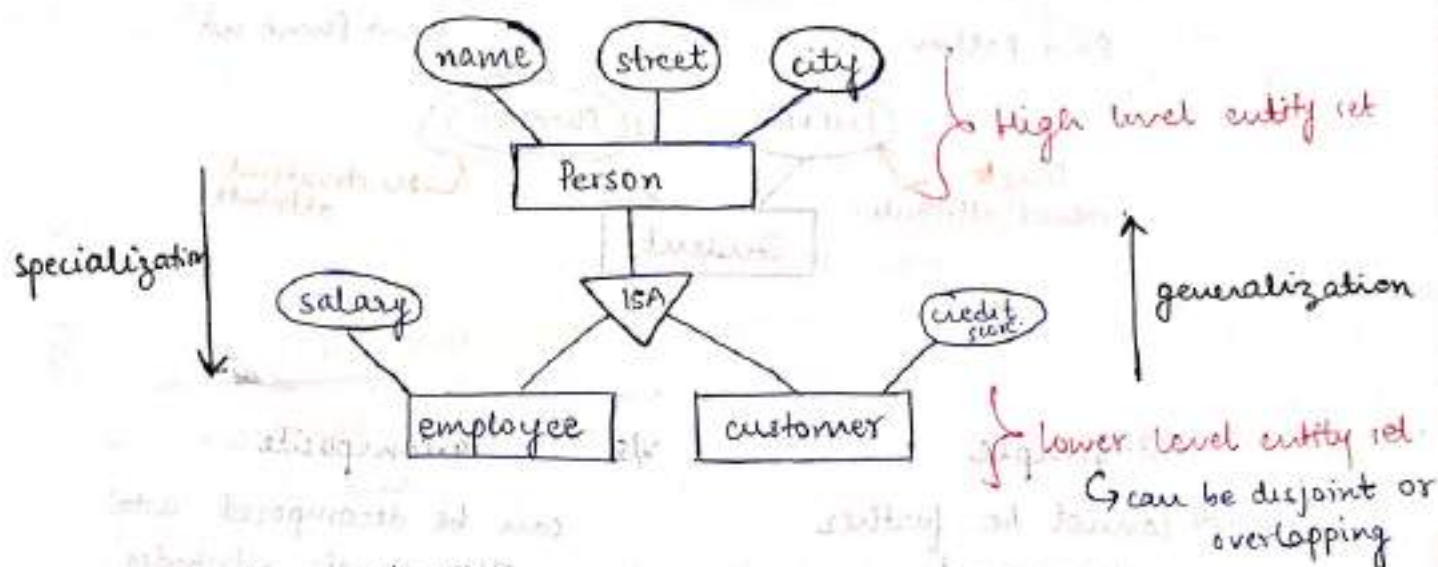
Strong entity - entity which has key attribute.



Extended ER Features

1. Specialization

The process of designating subgroupings within an entity set is called specialization.



2. Generalization

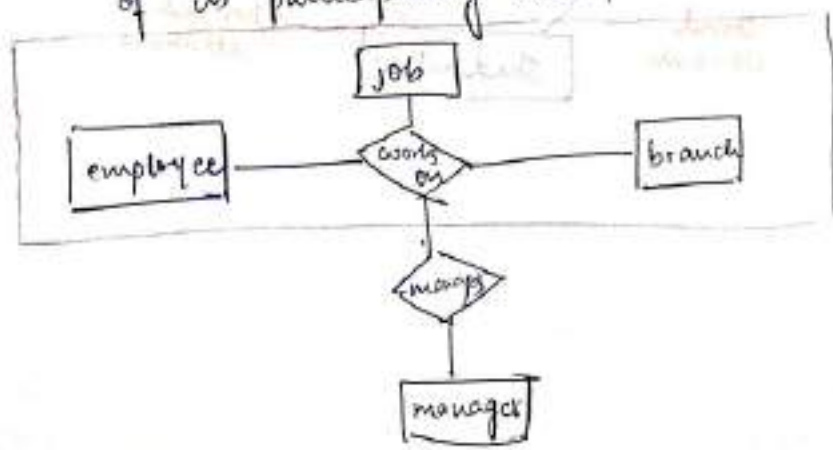
Commonality can be expressed using generalization. Containment relationship b/w higher level entity set & lower level entity set.

Total generalization / specialization - Each higher level entity set must belong to lower level entity set.

Partial generalization / specialization - Some members in higher level entity set may not belong to lower level entity set.

3. Aggregation

Relationship that contains another relationship as one of its participating entity



Types of attributes

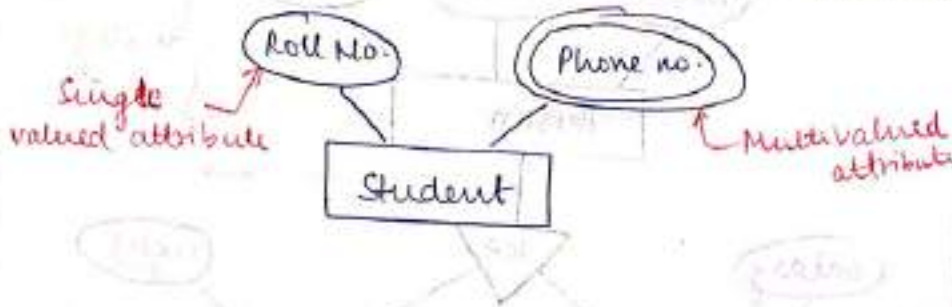
1. Single valued attributes can have only one value.

Ex → Roll no.

v/s

2. Multi valued attributes can have more than one value.

Ex → Phone no.



3. Simple

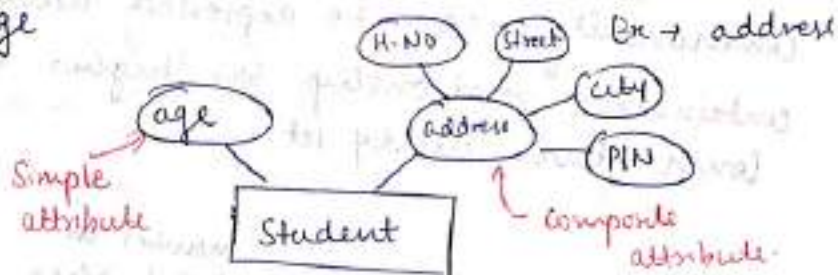
attribute cannot be further decomposed

Ex → age

v/s

4. Composite

can be decomposed into other simple attributes.



5. Stored

attributes whose value is actually stored in memory.

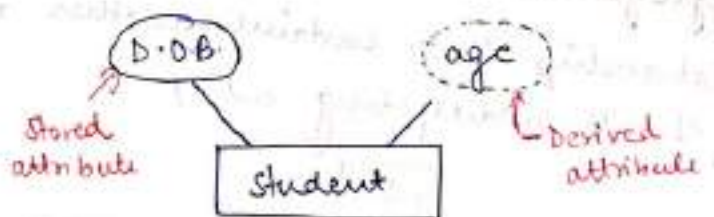
Ex → D.O.B.

v/s

6. Derived

attributes which can be derived from a given attribute.

Ex → age



7. Prime attribute which is part of some key

v/s

8. Non prime attribute which is not part of some key.

9. Complex attributes — Multivalued + Composite
Ex → address, phone no.

Relational model

The relational model uses a collection of tables to represent both data and relationships among data.

Database schema → logical design of database.

Relation_name (attribute₁, attribute₂, ..., attribute_n)

UNIQUE
NOT NULL

Domain constraint → specifies the important condition that we want each relation to satisfy.

Degree or Arity → number of attributes in a relation

Cardinality → number of tuples in a relation

NULL → representation of no value

Keys: attribute / group of attributes that can uniquely identify a tuple in a relation

Types of keys -

1. Super key - All possible keys of a relation.
superset of all keys.
2. Candidate key - Minimal superkey, it is called candidate key.
→ A key whose proper subset is not a key.
3. Primary key -
chosen from candidate key for implementation
4. Alternate key -
all candidate keys other than primary key
5. Foreign key -
key referring to some ~~alt~~ value in other table
can be NULL

Referential Integrity

The value in the foreign key should be present in the ~~tab~~ column of other table it is referring to

The foreign key should always refer to primary key

Functional Dependencies

Belief $A \rightarrow B$ (B is functionally dependent on A)
if each value in A is associated with exactly one value in B in the relation.

① A functional dependency $\alpha \rightarrow \beta$ always holds if α is the key to the relation R.
 $\alpha, \beta \rightarrow$ attributes / set of all attributes in R.

Closure of an attribute -
set of attributes that can be derived from the given attribute.

Ex $\rightarrow R(A, B, C, D)$

FDs $\rightarrow \{A \rightarrow B, A \rightarrow D, B \rightarrow C\}$

$A^+ = \{A, B, C, D\}$

$B^+ = \{B, C\}$

$C^+ = \{C\}$

$D^+ = \{D\}$

Ex $\rightarrow R(A, B, C, D, E)$

FDs $\rightarrow \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow E, E \rightarrow A\}$

$A^+ = \{A, B, C, D, E\}$

$B^+ = \{A, B, C, D, E\}$

$C^+ = \{A, B, C, D, E\}$

$D^+ = \{D, E, A, B, C\}$

$E^+ = \{E, A, B, C, D\}$

$\rightarrow$ Trivial functional dependencies

These functional dependencies are satisfied by all relations

Ex $\rightarrow A \rightarrow A$

$\alpha \rightarrow \beta$ if $\beta \subseteq \alpha$

Armstrong's axioms

$\rightarrow$ Reflexivity rule

$\alpha \rightarrow \beta$ holds if $\beta \subseteq \alpha$

$\rightarrow$ Augmentation rule

if $\alpha \rightarrow \beta$ holds, then $\gamma\alpha \rightarrow \gamma\beta$ also holds

$\rightarrow$ Transitivity rule

if $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$ holds, then, $\alpha \rightarrow \gamma$ also holds.

Additional Rules

→ Union Rules -

if $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$ hold, then,
 $\alpha \rightarrow \beta\gamma$ also holds.

→ Decomposition Rule -

if $\alpha \rightarrow \beta\gamma$ holds, then,
 $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$ also hold.

→ Pseudo transitivity rule.

if $\alpha \rightarrow \beta$ and $\gamma\beta \rightarrow \delta$, then,
 $\alpha\gamma \rightarrow \delta$ also holds.

Closure of a set of functional dependencies

Finding all possible FDs from given set of FDs

Ex: $R(A, B, C, D)$

FDs = $\{A \rightarrow B, B \rightarrow C, AB \rightarrow D\}$

$F^+ = \{A \rightarrow C, A \rightarrow D\}$

transitive
rule

→ pseudo transitive rule

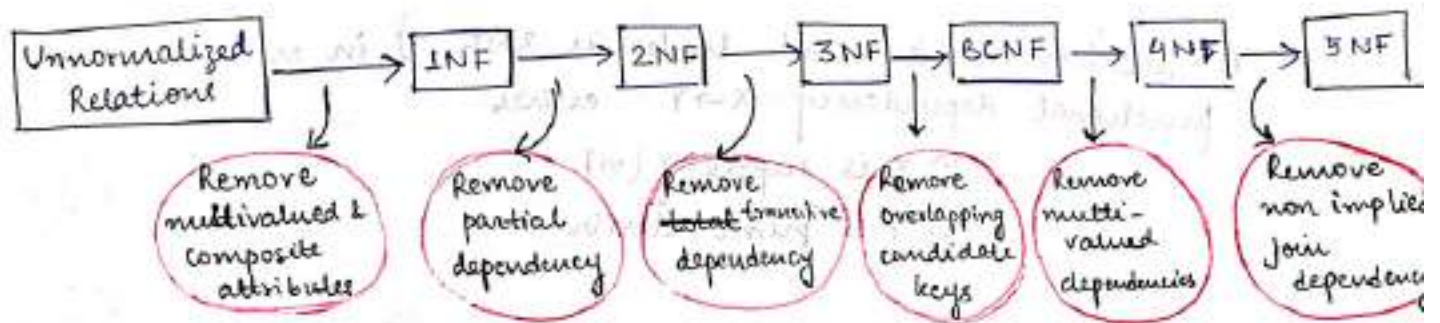
NORMALIZATION

Normalization is the process of minimizing redundancy from a relation or a set of relations

→ can cause insert, delete, update anomalies.

① Process of grouping attributes into well structured relations, that contain minimum redundancy.

Process of normalization —



First Normal Form (1NF)

A relation R is said to be in 1NF if there is no any multivalued & composite attribute

Second Normal Form (2NF)

→ Relation is in 1NF

→ no ^{non-}prime attribute in R should be partially dependent on R prime attribute.

Partial dependency :-

$X \rightarrow Y$ is partial dependency if some attribute A ∈ X can be removed and the dependency still holds.

→ If candidate key contains more than one attribute, then, a part of candidate key should not derive anything else.

- * No partial dependency if candidate keys contains only one prime attribute.

Third normal form (3NF)

- Relation R should be in 2NF
- No any non prime attribute in R should be transitively dependent on key of R.

A relation R is said to be in 3NF, if in every functional dependency $X \rightarrow Y$ either

- ⊙ X is superkey (or)
- ⊙ Y is prime attribute

Boyce Codd Normal Form (BCNF)

- Relation R should be in 3NF
- For every functional dependency $\alpha \rightarrow \beta$, α should be superkey.

* Functional dependency may not be preserved

* Any relation with 2 attributes is always into BCNF

* Normal form considered adequate for normal database design

Ans → 3NF.

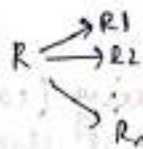
→ bcoz dependency preserving lossier guarantees.

Lossy Decomposition

The decomposition of relation R into $R_1, R_2, R_3, \dots, R_n$ is lossy when the join of $R_1, R_2, \dots, R_n$ does not produce same relation as in R .

< No of tuples inc are increased >

If no common attribute is present, then, the decomposition is lossy.



$$R = R_1 \bowtie R_2 \bowtie R_3 \dots \bowtie R_n$$

Lossless Decomposition

$$R \subset R_1 \bowtie R_2 \bowtie R_3 \dots \bowtie R_n$$

Lossy Decomposition

* Dep Decomposition is lossless when the common attribute is a key in one of the sub-relations.

* Lossless decomposition is guaranteed upto 3NF.

In BCNF, sometimes, it is possible to satisfy only lossless decomposition property or functional dependency preservation property.

We focus on making decomposition lossless.

$\therefore$ lossless decomposition holds in BCNF also.

when table is split into 2 parts Chase algorithm is used to check if decomposition is lossless or not

Dependency Preserving Decomposition

A decomposition $D = \{R_1, R_2, R_3, \dots, R_n\}$ of R is dependency preserving with respect to a set F of functional dependency if

$$(F_1 \cup F_2 \cup F_3 \dots \cup F_n)^+ = F^+$$

not ensured in BCNF

SQL: Structured Query Language

- ① Domain specific language
- ② used for managing data in RDBMS

→ SQL Data types -

CHAR (size) → a fixed length string (can contain letters, numbers, special characters)

$0 < \text{size} < 255$

Default → 1.

VARCHAR (size) → variable length string

Size defines the max. length of the string

$0 < \text{size} < 65535$

BIT (size) → a bit value type

BOOL → 0 is considered false, non zero value is true

INT (size) → medium integer

BIGINT (size) → large integer

FLOAT (size, d) → floating point no.

d → no. of digits after decimal point

size → total no. of digits

REAL (p) → floating point no.

p is used to determine whether to use FLOAT or DOUBLE

$0 < p < 24 \rightarrow \text{float}$

$25 < p < 53 \rightarrow \text{double}$

DOUBLE (size, d)

DECIMAL (size, d) → an exact fixed point no.

* SQL is not case sensitive

1. SELECT COMMAND

used to retrieve data from one or more tables.

SELECT <column-name> FROM <table-name>;

* returns
duplicates
if present.

SELECT * FROM <table-name>

→ returns whole table.

SELECT DISTINCT &del>country name FROM customers.

→ only returns unique records,
always used only with SELECT

SELECT DISTINCT country, postal FROM customers

distinct combination
of the 2 fields

2. WHERE Clause

① used with SELECT, UPDATE, DELETE & INSERT commands.

② used to filter specific rows from the table

SELECT * <column name> FROM <table-name>
WHERE <condition>

③ Relational operators that can be used in WHERE clause -

=, <>, <, <=, >, >=

④ Logical operators -

AND, OR, NOT

⑤ BETWEEN operator -

used to filter the records in specific range.

BETWEEN LB and UB

(both inclusive)



SQL Joins

- ① needed to retrieve records from more than one table collectively
- ② 2 tables must have one common column.

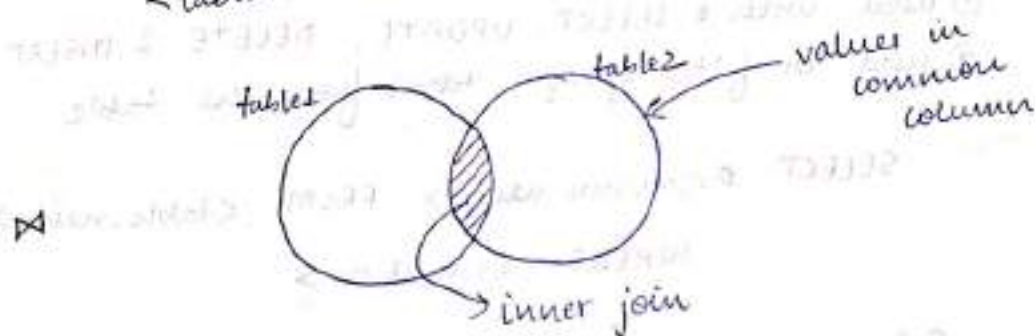
Types of joins —

- Full Join (Full outer join)
- Left Join (Left outer join)
- Right Join (Right outer join)
- Inner Join (Equi Join or Simple join)
- Self Join

No. of attributes in result = $\text{Deg}(\text{table1}) + \text{Deg}(\text{table2})$

1. Inner join

SELECT * FROM <table1> INNER JOIN <table2> ON
<table1>.<column> = <table2>.<column>



equivalent to

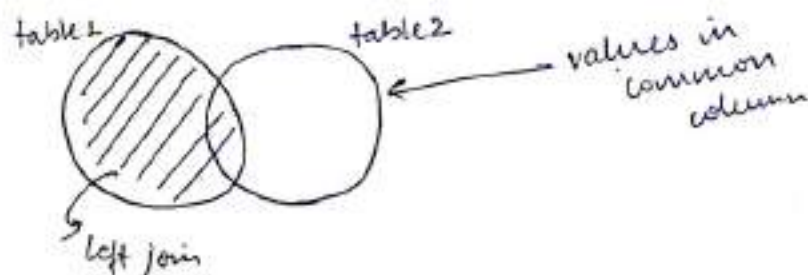
SELECT * FROM <table1>, <table2> WHERE
<table1>.<column> = <table2>.<column>

2. Left Join

SELECT * FROM <table1> LEFT JOIN <table2> ON
<table1>.<column> = <table2>.<column>

⌘

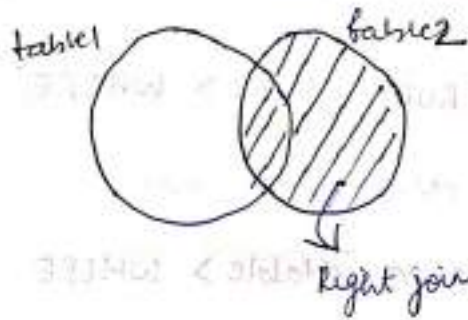
Common tuples + all tuples from table1



3. Right join

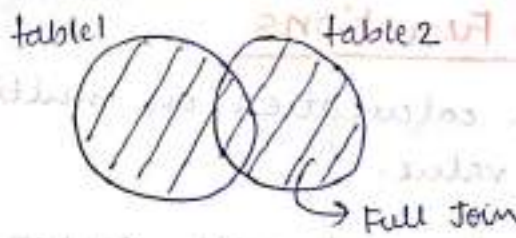
SELECT * FROM <table1> ~~<column>~~ RIGHT JOIN
<table2> ON <table1>.<column> = <table2>.<column>

common tuples + all tuples from table 2.



4. Full join

```
SELECT * FROM <table1> FULL JOIN <table2>
ON <table1>.<column> = <table2>.<column>
```

$$\text{common tuples} + \text{tuples from table 1} + \text{tuples from table 2}$$


5. Self join

only 1 table is involved in this join

self join is not keyword

∴ it is implemented using alias & WHERE clause

SELECT * FROM <table1> AS <t1>, <table1> AS <t2>
WHERE <table1> <t1>.<column1> = <t2>.<column1>

a	λ
14	1
14	2
14	10
14	5

LIMIT clause

used to limit the number of fetched records from a huge database table.

SELECT * FROM <table> WHERE <condition> LIMIT ^{no. of records needed} m, n

imp
*

SELECT * FROM <table> WHERE <condition> LIMIT m, n

The query returns max n records after leaving first m records

leave first m records

return n records

Aggregate Functions

- ① Performs a calculation on multiple values and returns a single value.
- * ② can only be used with SELECT clause

→ MIN(<column>) — returns min value in column

→ MAX(<column>) — returns max. value in column

→ SUM(<column>) — returns sum of values in column

→ COUNT(<column>) → returns no. of records

→ AVG(<column>) → returns average of values in column

all aggregate functions ignore NULL values except count(*)

A	B
1	N
2	N
N	N
3	N

Count(*) = 4

Count(A) = 3

Count(B) = 0

SUM(A) = 6

AVG(A) = 2

MAX(A) = 3

MIN(A) = 1

SUM(B) = NULL

AVG(B) = NULL

MAX(B) = NULL

MIN(B) = NULL

IN operator

① The IN operator allows you to specify multiple values in a WHERE clause.

② IN operator is shorthand for ~~many~~ multiple OR conditions.

SELECT * FROM <table> WHERE <column> IN (value1, value2, ...)

③ IN operators is also used with subqueries.

ALIAS

① used to give temporary name to a table, or a column of a table.

② only exists during duration of that query.

③ created using AS.

[space is not allowed in column name of a table.
To print space, ALIAS can be used]

SELECT CustomerName AS [Customer Name] FROM Customers.

GROUP BY Clause

① used in collaboration with SELECT statement to arrange identical data into groups.

② all attributes used in GROUP BY clause must be present in the SELECT clause.

③ attributes not present in GROUP BY clause can appear in aggregate functions.

SELECT DNo., AVG(salary) FROM Employees
GROUP BY DNo;

HAVING clause

similar to WHERE clause.

can only be used with ~~the~~ GROUP BY clause.

```
SELECT DNO, AVG (Salary) FROM Employees  
GROUP BY DNO HAVING Salary → AVG (Salary) > 50.
```

Order of execution → WHERE → GROUP BY → HAVING

WHERE

v/s

HAVING

checks the condition
on individual rows

checks the condition
on a group of rows.

~~can~~ cannot be used
with aggregate function

can be used with
aggregate functions

executed before
GROUP BY clause

executed after GROUP
BY clause.

Set operators

Set operators are used to combine results of two or more SELECT statements

no. of columns
returned
from the
individual
queries must
be equal.

(SELECT * FROM Table1) <set operator> (SELECT * FROM Table2)

- UNION (duplicate rows are eliminated)
- UNION ALL (duplicate rows are retained)
- INTERSECT (selects only common rows)
- MINUS / EXCEPT (set difference)

ORDER BY

- ① ORDER BY clause is used to sort the result set in ascending or descending order.

SELECT * FROM <table-name> ORDER BY <column-name> ASC/DESC
default

- ② default order of sorting → ascending order

- ③ ORDER BY clause is executed in the very end.

Subquery

- ① also called inner query, nested query

- ② query within a SQL query that is embedded within the WHERE clause

2 types of subquery

→ Single Row subquery

→ Multiple Row subquery.

~~Single Row subquery~~

- * Product with second highest price

SELECT MAX(price) FROM ~~Emo~~ Products

WHERE price < (SELECT max(price) FROM Products)

- * Product with third highest price

SELECT MAX(price) FROM Products

WHERE price < (SELECT max(price) FROM products

WHERE price < (SELECT max(price) FROM products));

Relational operators can be
used in single row subqueries

Multiple Row subquery

Operators used in Multiple Row subquery

1. IN operator

- ① allows to specify multiple values in WHERE clause.
- ② shorthand for logical OR.

SELECT * FROM Customers WHERE country IN
(SELECT country FROM suppliers);

* Find all customers who have placed more than 2 orders -
2 tables
Customers, Orders
SELECT * FROM Customers WHERE customerID IN
(SELECT customerID FROM orders
GROUP BY customerID HAVING count(customerID) > 2)

2. ANY operator

SELECT <column_name(s)>

FROM <table_name>

WHERE <column_name> <operator> ANY

(subquery) (any standard comparison operator

(= < > <= >= < >)

3. ALL operator

SELECT <column_name(s)>

FROM <table_name>

WHERE <column_name> <operator> ALL

(subquery)

4. EXISTS operator

- ⊙ used to test for the existence of any record in a subquery.
- ⊙ returns true if subquery returns one or more records.

```
SELECT <column_name(s)>  
FROM <table_name>  
WHERE <column_name> EXISTS  
      (subquery)
```

* 'SELECT NULL' returns 1 record with column & row NULL

NULL
NULL

RELATIONAL ALGEBRA

- Relational algebra is a procedural query language which takes instances of relations as i/p & yields instances of relations as o/p.
- provides theoretical foundation for relational databases

Basic operators—

- | | |
|--------------------------|----------------------------------|
| ⊙ Select (σ) | ⊙ Intersection ($\cap$) |
| ⊙ Project (π) | ⊙ Cartesian Product ($\times$) |
| ⊙ Union ($\cup$) | ⊙ Rename (ρ) |
| ⊙ Set Difference ($-$) | ⊙ Division ($\div$) |

1. Selection operator

→ fetches tuples which satisfy a condition

$$\sigma_{\langle \text{conditions} \rangle} (\langle \text{Relation_name} \rangle)$$

→ similar to WHERE clause of SQL.

① Selection operator is commutative.

$$\begin{aligned}\sigma_{\langle \text{cond1} \rangle} (\sigma_{\langle \text{cond2} \rangle} (R)) &= \sigma_{\langle \text{cond2} \rangle} (\sigma_{\langle \text{cond1} \rangle} (R)) \\ &= \sigma_{\langle \text{cond1} \rangle \wedge \langle \text{cond2} \rangle} (R)\end{aligned}$$

2. Projection operator

→ used to select only certain columns of the table.

$$\pi_{\langle \text{column_name(s)} \rangle} (R)$$

* eliminates duplicates in result.

→ similar to SELECT statement SELECT DISTINCT statement.

RELATIONAL ALGEBRA

3. Set operations

① The 2 relations on which set operations are applied should necessarily have similar data types of tuples.

② they should be union compatible

* duplicates are not present.

- Commutative & associative {
- i) Union operator ($\cup$)
 - ii) Intersection operator ($\cap$)
 - iii) Set Difference ($-$)

$$* \pi_x(R_1 \cup R_2) = \pi_x(R_1) \cup \pi_x(R_2)$$

$$* \pi_x(R_1 \cap R_2) \neq \pi_x(R_1) \cap \pi_x(R_2)$$

4. Cartesian Product

cross product

$$|R \times S| = m \times n$$

$$|R| = m$$

$$|S| = n$$

$$\text{Deg}(R \times S) = \text{Deg}(R) + \text{Deg}(S)$$

→ Joins

i) Conditional join

$$R_1 \bowtie_{\langle \text{condition} \rangle} R_2$$

ii) Equi join

$$R_1 \bowtie_{R.C = S.C} S$$

iii) Natural join

Both relations should have atleast one common column name

$$R \bowtie S$$

iv) Left outer join

$$R \ltimes S$$

v) Right outer join

$$R \ltimes S$$

vi) Full outer join

$$R \ltimes S$$

5. Rename operator

⊙ similar to aliasing in SQL.

⊙ used to give temporary name to relation / attribute

$$\rho_{\langle \text{newname} \rangle}(\langle \text{relation} \rangle) \rightarrow \text{Rename relation}$$

$$\rho_{R(A,B,C)}(R) \rightarrow \text{Rename columns to A, B, C}$$

6. Division operator

$$\odot R_1 \div R_2$$

Attribute set of $R_2 \subseteq$ Attribute set of R_1 .

$$\text{Result set} = (\text{Attribute set of } R_1) - (\text{Attribute set of } R_2)$$

R_1

A	B	C

R_2

A	B

$R_1 \div R_2 \rightarrow$ Those values of C which are associated with every value of A, B in R_2

< used in those queries which contain 'for all' >

TRANSACTION

logic unit of database which includes one or more database operations

→ Commit - command used to make the changes permanent in a database. (operation successful)

→ Rollback - command used to revert back to the original state of database (operation unsuccessful)

Database state becomes same as it was on last commit

ACID Property of Transaction

1. Atomicity:- All or none.
(either complete transaction or never start it)

2. Consistency:- the database state must be consistent

3. Isolation:- transactions should execute such that it does not affect nor is affected by any other concurrent transaction

4. Durability:- changes made should be reflected back into database (permanently)

Schedule

Execution ^{order} of multiple transactions on a same database concurrently is called schedule.

Why concurrency?

1. Improved throughput (No. of lines executed per unit time)
2. Resource utilization.
3. ~~Increase~~ ^{Reduced} waiting time.

Problems with concurrency-

1. Recoverability problems
2. Deadlock
3. Serializability issues.

Serializability problems

- Lost update problem
- Dirty Read problem
- Phantom tuple
- Unrepeatable Read problem
- Incorrect summary problem.

Good schedule - concurrent schedules which provide consistent result.

Bad schedule - These schedules do not give consistent result (suffer from one or more of the above problems)

Schedule

Serial schedules

- ① schedules which do not have interleaving
- ② always give consistent result
- ③ For n transactions, no. of serial schedules = $n!$

Non serial schedules

- ① schedules which have interleaving are called non serial schedule.

$T_1 \rightarrow n_1$ steps

$T_2 \rightarrow n_2$ steps

$\vdots$

$T_n \rightarrow n_n$ steps

NO of schedules possible

$$= \frac{(n_1 + n_2 + n_3 + \dots + n_n)!}{n_1! n_2! n_3! \dots n_n!}$$

Serializable schedule

- ① A concurrent schedule which provides result as a serial schedule is called serializable schedule.

Serializability

- Conflict Serializability
- View Serializability

1. Conflict Serializability

→ 2 DB statements have conflict if:

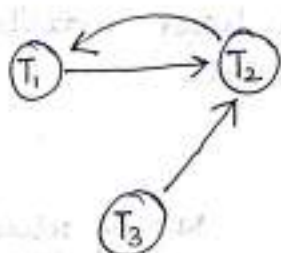
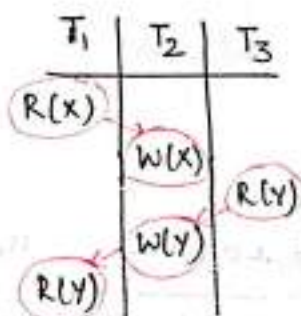
- Both are in different transactions
- Both are accessing same data item
- One of them is write operation

→ 2 transactions are conflict equivalent if the conflicting statements appear in the same order in both of them.

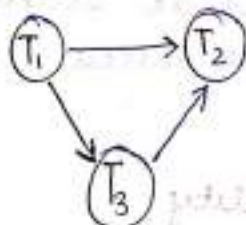
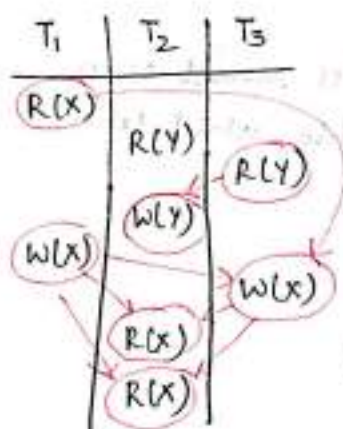
→ If one of the 2 transactions is serial schedule, the other one is called conflict serializable schedule.

To prove if a given schedule is conflict serializable, we use precedence graph

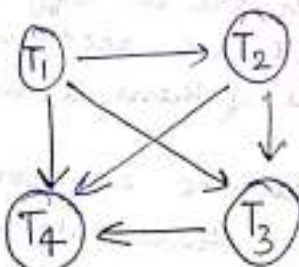
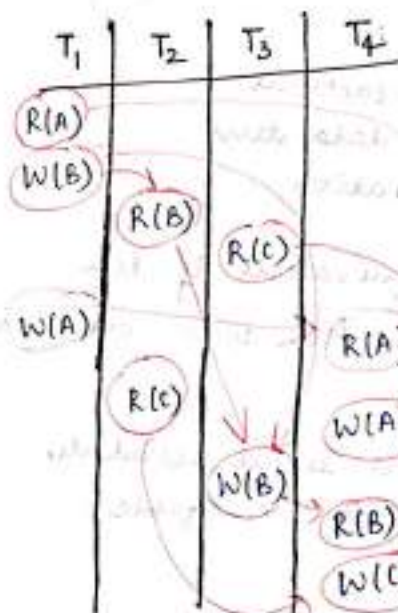
vertices $\rightarrow$ transactions
 edges $\rightarrow$ conflicts
 directed graph



$\therefore$ cycle is present in precedence graph,
 $\therefore$ given schedule is not conflict serializable.



no cycle
 $\therefore$ conflict serializable schedule
 eg. serial schedule = T₁ T₃ T₂



conflict serializable schedule

eg. conflict serializable
 eg. serial schedule —

T₁ T₃ T₄ T₂
 T₁ T₃ T₂ T₄
 T₁ T₂ T₃ T₄
 T₂ T₁ T₃ T₄

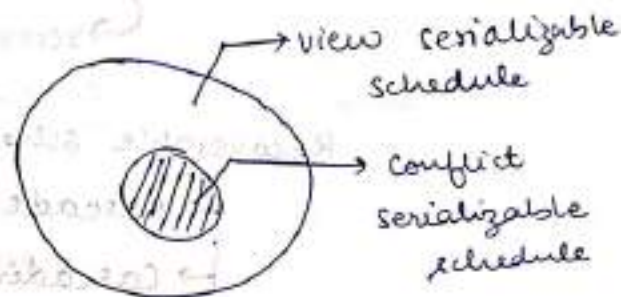
2. View Serializability

3 points to remember

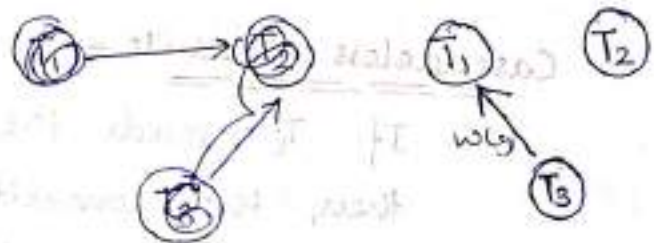
- ↳ who reads from database
- ↳ who reads from other's written value
- ↳ who writes last

② 2 schedules S_1 & S_2 are view equivalent if both are following similar sequence/rules of view serializability.

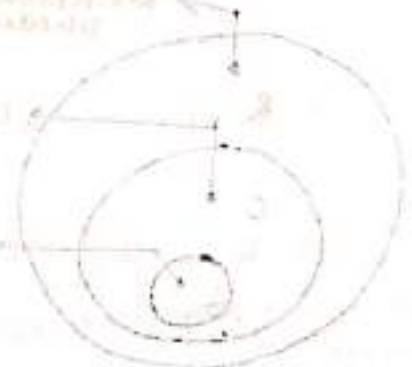
③ All conflict serializable schedules are view serializable.



T_1	T_2	T_3
$R(X)$	$R(Y)$	$w(Y)$
$w(Y)$	$w(X)$	



not view serializable. $\begin{matrix} T_3 \\ T_3 \\ T_2 \end{matrix}$ $\begin{matrix} T_1 & T_2 \\ T_2 & T_1 \\ T_3 & T_1 \end{matrix}$ } 3 possible serial schedules



Recoverability

When no any committed transaction needs to roll back, it is called recoverable schedule

If a transaction T_i reads dirty value written by T_j and then, the commit operation of T_i should be after the transaction T_j commits.

Recoverable schedule.

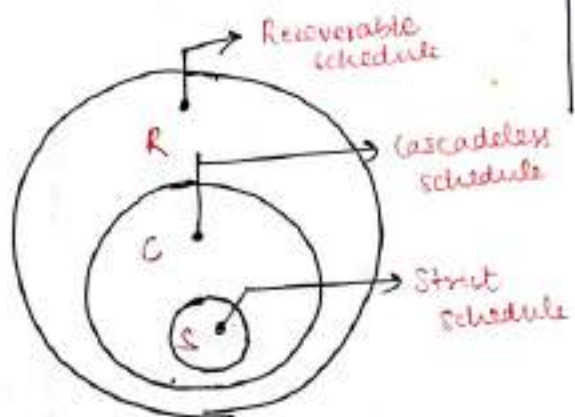
Recoverable schedule

- Cascadeless Recoverable schedule Rollback
- Cascading Recoverable schedule Rollback

Cascadeless schedule -

If T_i reads the data item written by T_j then, the commit operation of T_j should appear before read operation of T_i

T_j	T_i
$R(x)$	
$x = x + 2$	
$w(x)$	
commit	
	$R(x)$
	$x = x + 3$
	$w(x)$
	commit



Obtaining serializability (view serializable & conflict serializable) is not practically possible $\therefore$ locking protocols are used.

Locking Protocols

Lock $\begin{cases} \text{Shared lock (other transactions can share)} \\ \text{taken for read} \\ \text{Exclusive lock (no other transaction can read/write)} \\ \text{taken for write.} \end{cases}$

Demand currently held	Shared	Exclusive
Shared	✓	X
Exclusive	X	X

* A transaction may suffer from starvation if it demands exclusive lock while shared lock is being held by multiple transactions.

low to avoid starvation? * If a transaction is blocked for exclusive lock because other transactions have shared lock, then multiple shared locks are not allowed.

Availability	Mode	Count	Blocked
X			

Table maintained by DBMS

0 if available
1 if lock is held.

S $\rightarrow$ shared
EX $\rightarrow$ Exclusive

No. of transactions holding lock (only in case of shared lock)

Transactions (FIFO order) that are waiting for the data item to be unlocked.

Simple locking mechanism can lead to unrepeatable read problem.

T ₁	T ₂
Lock-S(x) R(x) Unlock(x)	Lock-X(x) W(x) Unlock(x)
Lock-S(x) R(x)	

$\therefore$ To solve unrepeatable read problem, several protocols ^{rules} are developed.

2 Phase Locking Protocols (~~2PL~~ Basic 2PL)

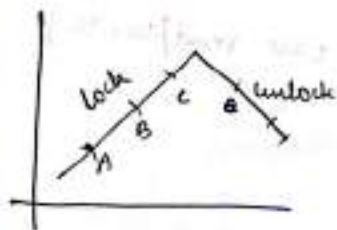
→ Systematic Locking mechanism

→ Once unlock is done, a transaction is not allowed to lock any other data item.

Each transaction has 2 phases —

→ Growing phase - acquiring of locks

↳ Shrinking phase - releasing of locks.



T_1	T_2
LOCK-S(X)	
R(X)	
LOCK-EX(Y)	
Unlock(X)	
	LOCK-EX(X)
	W(X)
W(Y)	
Unlock(Y)	
	LOCK-S(Y)
	R(Y)
	Unlock(X)
	Unlock(Y)

$\therefore$ Only 2 possible schedules are there —

T_1	T_2
$R(x)$	$W(x)$
$W(y)$	

91

T_1	T_2
$R(x)$	
$w(y)$	$w(x)$ $w(y)$

T_1	T_2
$R(x)$	$w(x)$
$w(y)$	$R(y)$

schedule
not
allowed
by basic
2PL.

▲ Every schedule which is allowed under basic 2PL is conflict serializable also.

Question asked - Tell whether the given schedule is allowed under 2PL or not.

★ 2PL suffers from starvation.

* 2PL suffers from deadlock

T_1	T_2	T_3	T_4
$w(x)$	$w(y)$	$w(z)$	

$R(x)$
 $R(y)$
 $R(z)$

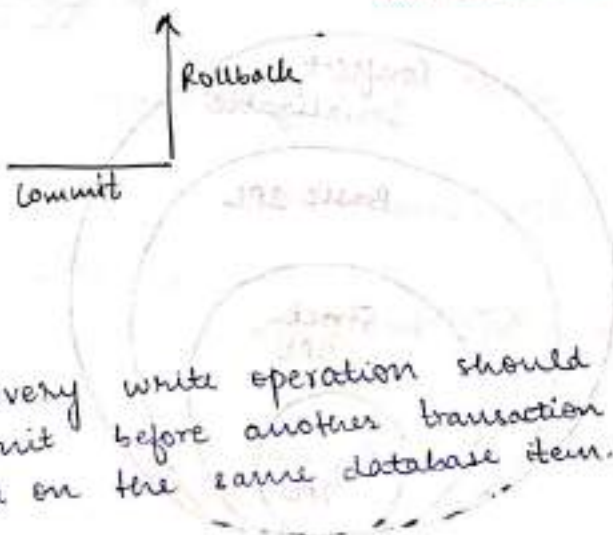
T_1	T_2
LOCK-S(X) R(X)	LOCK-S(Y) R(Y)
LOCK-EX(Y) W(Y)	LOCK-EX(X) W(X)

* 2PL can allow non recoverable schedules

T_1	T_2
	LOCK-EX(X)
	R(X)
	W(X)
	Unlock(X)
LOCK-EX(X)	
R(X)	
W(X)	
Unlock(X)	
Commit	

Schedule is allowed under basic 2PL

not recoverable.



Strict Schedule - Every write operation should be ended with commit before another transaction performs read/write on the same database item.

Strict 2PL Protocol

Exclusive locks should be released only after commit.

* Strict 2PL allows only strict schedules.

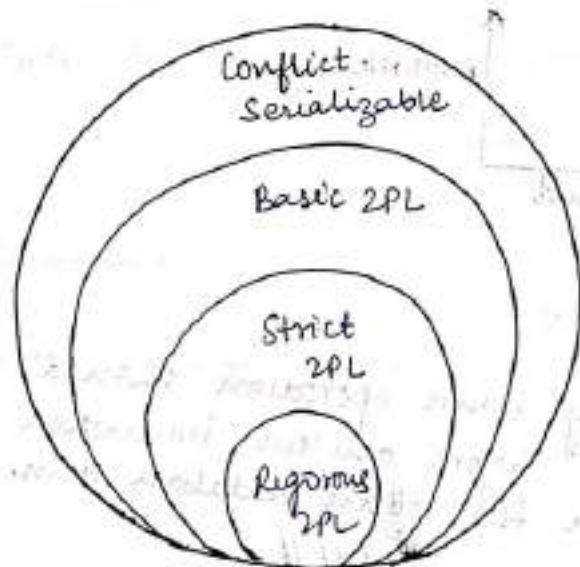
Rigorous 2PL Protocol

Every lock should be released after commit.

- ① Strict & Rigorous 2PL do not suffer from dirty read problem.
- ② Basic 2PL may allow non recoverable schedule.
- ③ Strict & Rigorous 2PL have only recoverable schedules (cascadeless)

Conservative 2PL Protocol

Lock all the ~~trans~~ items before the transaction begins its execution by predeclaring read-set & write-set



Timestamp Based Protocols

Timestamp:- unique identifier created by DBMS to identify the relative starting time of a transaction

$TS(T_1) < TS(T_2) \rightarrow T_1$ starts before T_2

T_1 is older than T_2

Read Timestamp(A) $\rightarrow$ Youngest transaction who reads A

Write Timestamp(A) $\rightarrow$ Youngest transaction who writes A

T_1	T_2	T_3
R(A)	-----	-----
R(A)	-----	-----
		R(A)
R(A)		

*youngest & latest

$R_TS(A) = 1$

$R_TS(A) = 1$

$R_TS(A) = 3$

$R_TS(A) = 3$ (because T_3 is youngest)

Basic Timestamp algorithm is used to control concurrency by allowing transactions to run in their relative ordering of timestamps.



When a transaction T issues $w(x)$ operation, check the following conditions -

① If $R_TS(x) > TS(T)$ or $W_TS(x) > TS(T)$ abort the transaction and rollback T .

② else, execute $w(x)$ and update $W_TS(x)$ to $TS(T)$.

When transaction T issues $R(X)$ operation, check the following conditions -

- ① If $W_{TS}(X) > TS(T)$ then abort and reject the operation.
- ② else execute $R(X)$ and set $R_{TS}(X)$ to $\max(TS(T), R_{TS}(X))$

Thomas White Rule

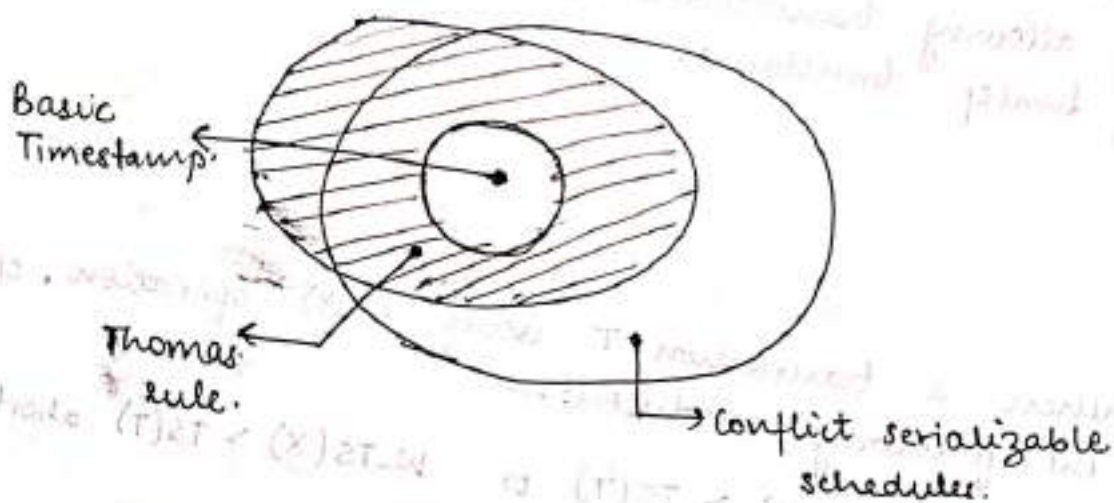
① Read is same as basic timestamp rules.

② $W(A)$ in transaction T :

→ If $R_{TS}(A) > TS(T)$, abort T , rollback

→ else if $W_{TS}(A) > TS(T)$, skip the write operation

→ else perform $W(A)$ and update $W_{TS}(A) = TS(T)$



③ Some schedules which follow Thomas white rule are not conflict serializable

Deadlock Prevention

~~1. Wait-Die~~

Let T_i = older transaction

T_j = newer younger transaction

$$TS(T_i) < TS(T_j)$$

1. Wait Die -

→ If T_i tries to acquire lock currently held by T_j , then, T_i waits till the lock is available.

→ If T_j tries to acquire lock currently held by T_i , then, T_j is aborted and restarted with same time stamp.

2. Wait Wound -

→ If T_i tries to acquire lock currently held by T_j , then, T_j is pre-empted by aborting it.

→ If T_j tries to acquire lock currently held by T_i , then, T_j waits till the lock is available.

Disk Block -

- ① logical unit of storage created by file system.
- ② A combination of 1-2 sectors.
- ③ size = 512B to 2KB



Record storage -

Suppose, 1 disk block = 64 bytes = 2^6 bytes

Student

Name	Roll No.	...

→ 1 record
= 4 bytes

∴ No. of records in 1 disk block = $\frac{2^6}{2^2} = 2^4 = 16$

If the table contains 512 records,

no. of disk blocks needed = $\frac{512}{16} = \underline{\underline{32 \text{ blocks}}}$

Fixed v/s Variable Length Records -

Student table

Roll No.	Name	Photo (max. 2MB)	Sign (max. 200kb)

max. size of 1 record $\approx 2.2 \text{ MB}$

If a student uploads pic of size 1 MB, sign 200 kb,

size of record = 1.2 MB.

In GATE syllabus → In fixed length record, size of record is fixed = 2.2 MB (extra space is wasted)

In variable length record, size of record is not fixed (space not fully utilized)

Spanned Mapping -

record of a file is stored inside the block even if it can only be stored partially, hence, record is spanned over multiple blocks.



Adv:- no wastage of memory (internal fragmentation)



Disadv:- memory access time is large.

Block size = 32B

Record size = 5B

6 records in block 1

2B of 7th record in b1

3B of 7th record in b2

Unspanned Mapping

a record of file is stored inside a block only if it can be completely stored inside it.



Adv:- access time of records is less



Disadv:- wastage of memory (internal fragmentation)

<space wasted per block>

Block size = 32B

Record size = 5B

6 records in B1

7th records in B2

2B in B1 wasted

Indexing

index file is created to store information of which records are present in which disk block.

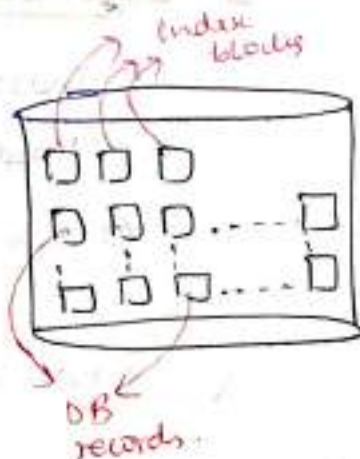
Index file/table is also stored in disk blocks.

→ Dense index:

Index record is for each database record.

→ Sparse Index:

Index record is for a few records only.



Indexing Techniques

- Primary indexing
- Clustering indexing
- Secondary key indexing
- Secondary non key indexing

1. Primary indexing

- ① Indexing is done on key primary key or super key.
- ② Data must be ordered on index.
- ③ It's always sparse index.

जिस order में index है,
उसी order में data भी
होना चाहिए

$$\text{One index record} = \text{Indexed column value} + \text{Block pointer}$$

2. Clustering indexing

- ① Indexing is done on non key field. (duplicate values may be present)
- ② Data must be ordered on index field

→ Indexing is done for unique value of non-key

- ③ sparse indexing

3. Secondary key Indexing

- ① Indexing done on primary key or any super key
- ② Data must not be ordered on index field
- ③ can be dense or sparse

< non key > → similar to key indexing
except that indexing is done on
non key field.

B Tree

- ① Tree based indexing technique.
- ① Dynamic indexing technique.
- ① Self balancing ~~two~~ search tree

→ grows horizontally as well as vertically.

Order of B tree - max children of a node of B Tree
(order is denoted by 'p')

An order p B-Tree -

↳ every node has max p children. (tree pointers)

↳ every node other than root should have atleast $\lceil \frac{p}{2} - 1 \rceil$ nodes.

↳ every node can have atmost (p-1) keys and p tree pointers

↳ Root can have minimum 1 node key

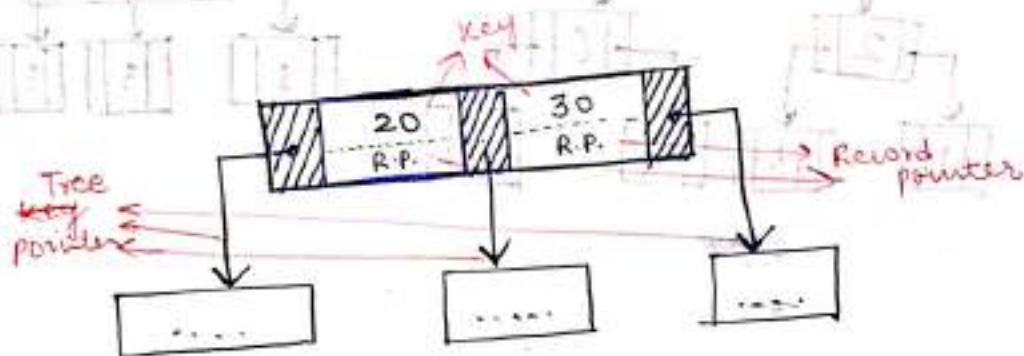
↳ all leaves are at same level.

B Tree node structure

Key - index pointer

Record pointer:- pointer to the disc block where ~~the~~ key index is stored.

Tree pointer:- children.



Example -

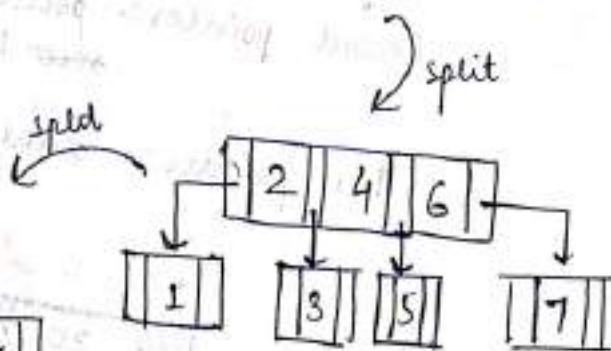
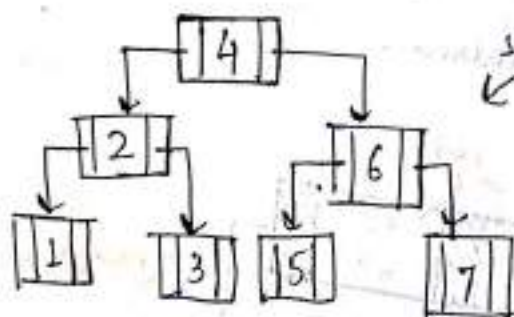
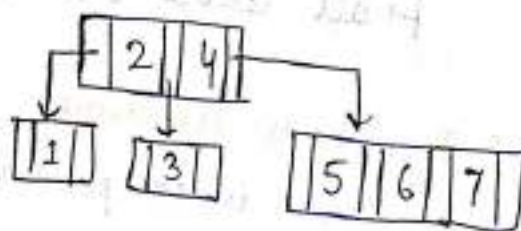
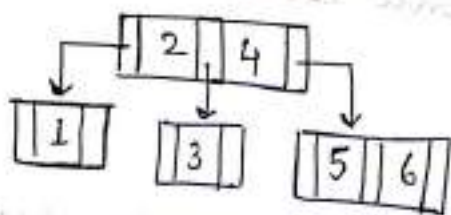
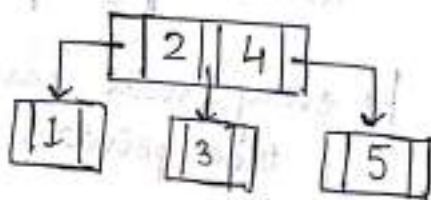
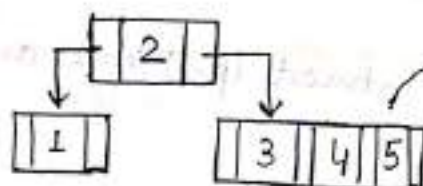
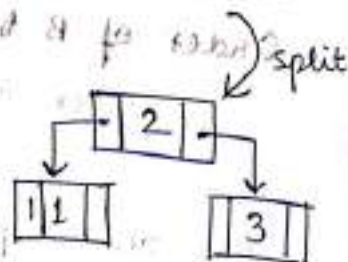
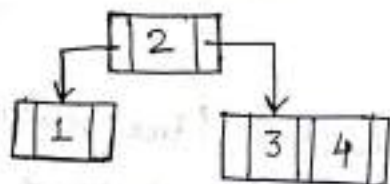
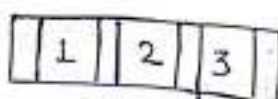
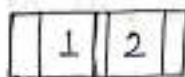
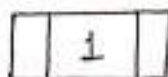
order 3 BTree

Max keys = 2

Max links (children) = 3

Min keys (except root) = $\lceil \frac{3}{2} \rceil - 1 = 1$
 Max keys (except root) = $\lceil \frac{3}{2} \rceil = 2$

Insert keys 1, 2, 3, 4, 5, 6, 7



Each node of a B Tree is stored in a disk block.

If p is order of tree,

max No. of keys = $p-1$

max. No. of record pointers = $p-1$

max. No. of tree pointers = p .

formula
used to find

max. value of 'p'

if block size is given.

or a min block size if p is given.

$$\therefore (p-1)(\text{key size}) + (p-1)(\text{record pointer size}) + p(\text{tree pointer size}) \leq \text{Block size}$$

Imp. -

p order B tree

Total nodes = n

$H_{\min} = \text{min height of BTree} = \lceil \log_p (n+1) - 1 \rceil$

$H_{\max} = \text{max height of BTree} = \lceil \log_{\lceil \frac{p}{2} \rceil} \frac{n+1}{2} \rceil$

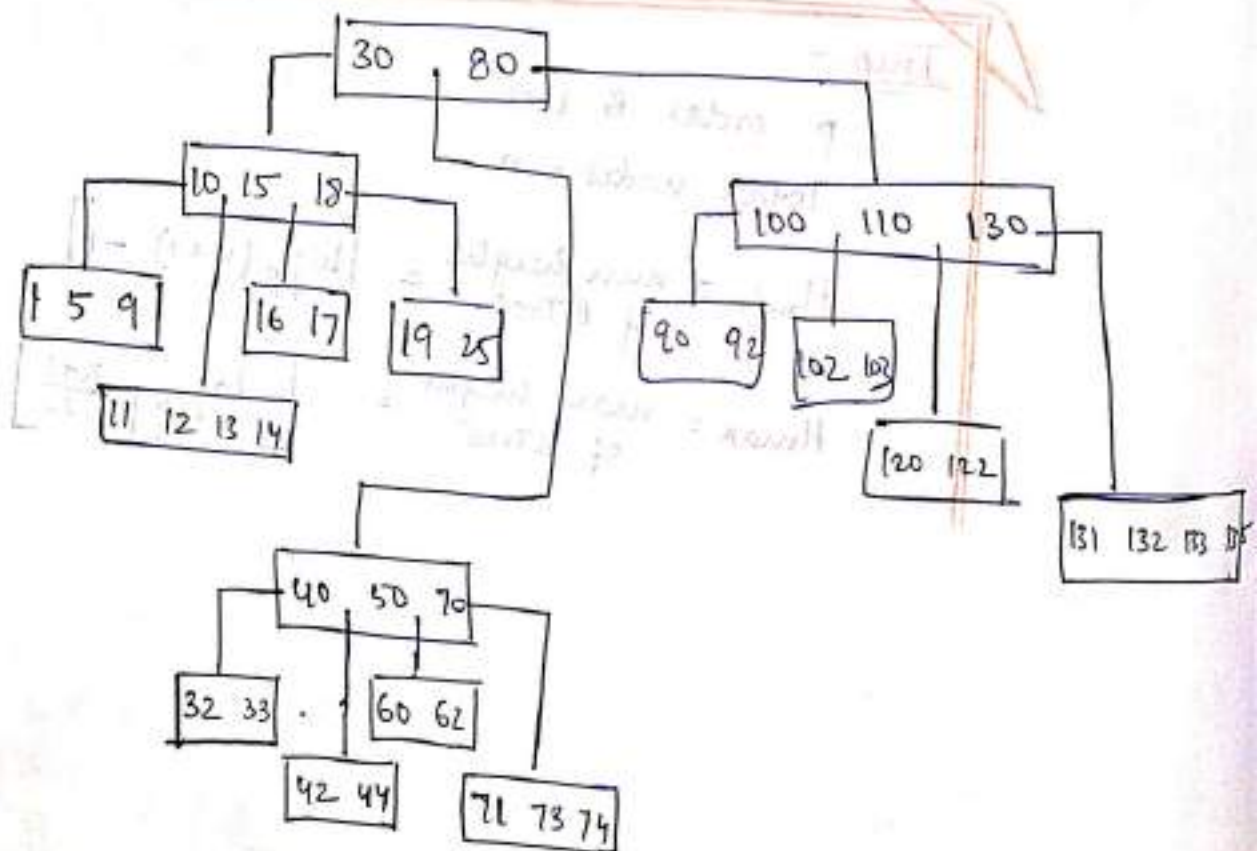


- B-Tree deletion in leaf node -
- After deletion, if no violation of ~~new~~ minimum keys then no changes in tree.
 - If violation of min keys, then, borrow key from sibling (rotation through parent)
 - If borrow through parent can't be possible then merge the node with sibling & pull down the ~~set~~ anchor ~~node~~ ^{key} from parent.

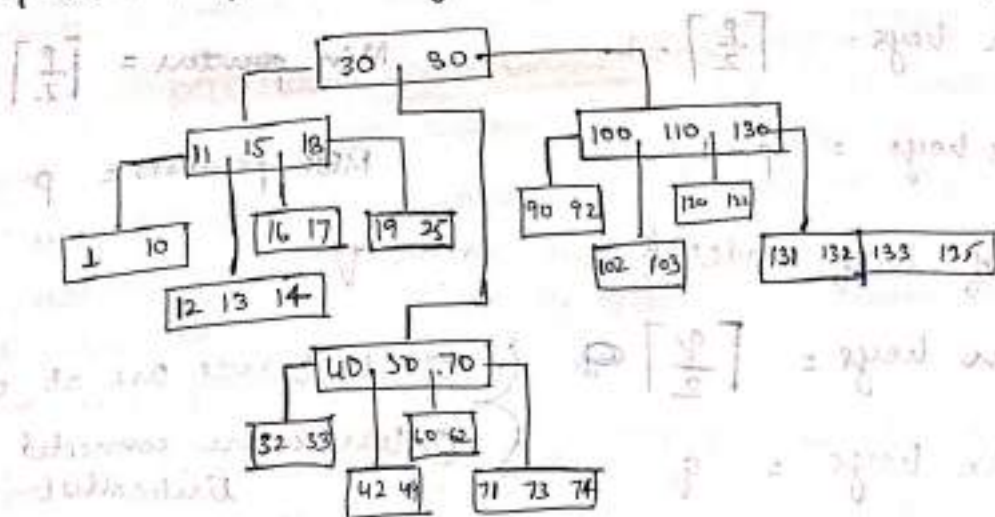
B-tree of order 5

max keys = 4

min keys = $\left\lceil \frac{5}{2} \right\rceil - 1 = 2$



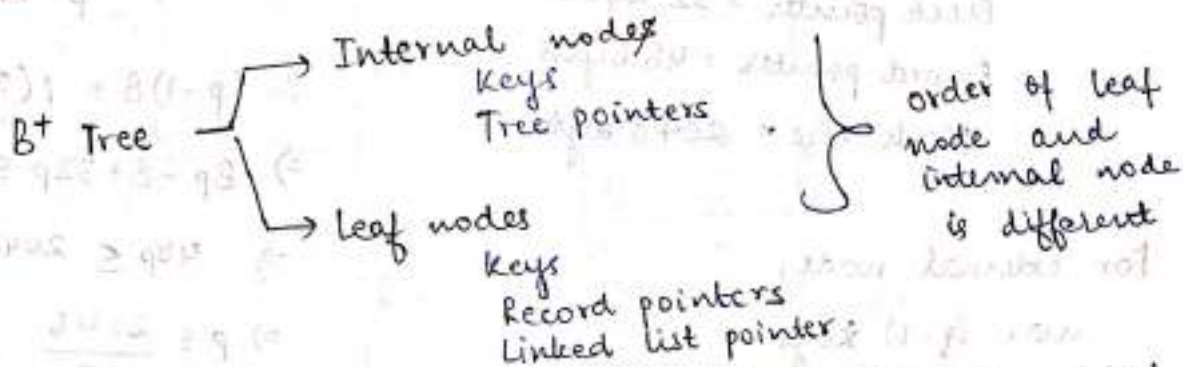
Delete 5 → no change
 Delete 9 → borrow from sibling



B tree deletion from internal node -

- Replace the deleted value with inorder successor or inorder predecessor.
- Now follow rules of deletion of key from leaf node.

B⁺ Tree



① Leaf nodes are connected to each other ~~thru~~ as linked list

① Range queries work faster in B⁺ Trees.

Order of internal nodes (except root) $\rightarrow$ order p .

$$\text{Min keys} = \left\lceil \frac{p}{2} \right\rceil - 1 \quad \text{Min pointers} = \left\lceil \frac{p}{2} \right\rceil$$

$$\text{Max keys} = p - 1 \quad \text{Max pointers} = p$$

Order of leaf nodes $q \rightarrow$ order q

$$\text{Min keys} = \left\lceil \frac{q}{2} \right\rceil$$

Min keys = $\left\lceil \frac{q}{2} \right\rceil$ $\left\{ \begin{array}{l} \text{① all leaves are at same level} \\ \text{② leaves are connected using linked list.} \end{array} \right.$

$$\text{Max keys} = q$$

* Splitting at internal node is same as that of BTree
Splitting at leaf node - middle key is duplicated & is part of right subtree.

① One node of B+ tree must be present in one disk block.

order of B+ tree = p

Key = 8 bytes

Block pointer = 32 bytes

Record pointer = 48 bytes

Block size = 2048 bytes

For internal node,

max $(p-1)$ keys
 p block pointers

$$\therefore (p-1)8 + p(32) \leq 2048$$

$$\Rightarrow 8p - 8 + 32p \leq 2048$$

$$\Rightarrow 40p \leq 2048 + 8 \leq 2056$$

$$\Rightarrow p \leq \frac{2046}{40} \quad 51.125$$

$$\therefore p_{\max} = 51 \text{ (max)}$$

max order of internal nodes of B+ tree = 51

$$\therefore (q-1)(8) + (q-1)(48) \leq 2048$$

$$\Rightarrow (q-1)(56) \leq 2048$$

$$\Rightarrow q-1 \leq \frac{2048}{56}$$

$$\Rightarrow q-1 \leq 36.57 \Rightarrow q \leq 37.57$$

$$\therefore q_{\min} = 37 \text{ (max)}$$

max order of leaf node of B+ tree = 37

~~Multipl.~~

Multi-level indexing

→ In secondary key indexing, if the no. of records in the database table is huge, then, the index file size will also be very huge. & searching will need to access multiple disk blocks.

→ entire index file cannot be stored in a single disk block.

→ ∴ Do indexing on the index file.
< Primary indexing >

No. of blocks accesses needed to search a record is minimized.

